

PROJECT REPORT

STUDENT DETAILS:

Name: Rishma RM Ashmi

Roll.no: 24f2003378

Email: 24f2003378@ds.study.iitm.ac.in

About me: I am currently pursuing the Diploma Level in the Indian Institute of Technology Madras BS Program and am also a second-year B.Tech Computer Science Engineering student.

I have a strong interest in software development and web technologies, with a focus on building practical and efficient applications.

Through this project, I have strengthened my skills in backend development, database management, and system design.

PROJECT DETAILS:

Project title: Placement Management System

Problem Statement: To design and develop a web-based Placement Management System that allows administrators, companies, and students to manage, track, and streamline placement activities such as student registrations, company postings, and application status updates efficiently.

Approach: The application was developed using Flask as the backend framework with a modular structure to ensure scalability and maintainability. It provides role-based access for admin, companies, and students to manage registrations, job postings, and application tracking efficiently. The system integrates a database for secure data storage and enables real-time status updates to streamline the placement process.

AI/LLM DECLARATION:

I used AI to assist in drafting project documentation, refining problem statements, and improving code structure suggestions.

The extent of AI/LLM usage is approximately 10–15%, limited to guidance on formatting, minor code improvements, and clarity enhancements.

All core implementation, database integration, debugging, and system design were completed independently.

TECNOLOGIES AND FRAMEWORK USED:

Technology / Library	Purpose
Flask	Core backend web framework
SQLite	Lightweight database for storing student, company, and application data
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive user interface design
Flask-Login	User authentication and session management
WTForms	Form handling and validation
HTML5 & CSS3	Structuring and styling web pages
Python	Core programming language used for backend logic

DATABASE SCHEMA / ER DIAGRAM:

Tables:

1. Admin — stores administrator credentials (id, email, password)
2. Student — stores student details (id, name, email, password, department, contact, cgpa, resume, active)
3. Company — stores company information (id, name, email, password, website, location, hr_contact, approved, active)
4. Drive — stores placement drive details (id, role, eligibility, etc, deadline, description, approved, closed, company_id)
5. Application — stores student applications for drives (id, status, applied_on, student_id, drive_id)

Relationships:

1. One-to-Many → Company → Drive

One company can create multiple placement drives.

2. One-to-Many → Student → Application

One student can apply to multiple drives.

3. One-to-Many → Drive → Application

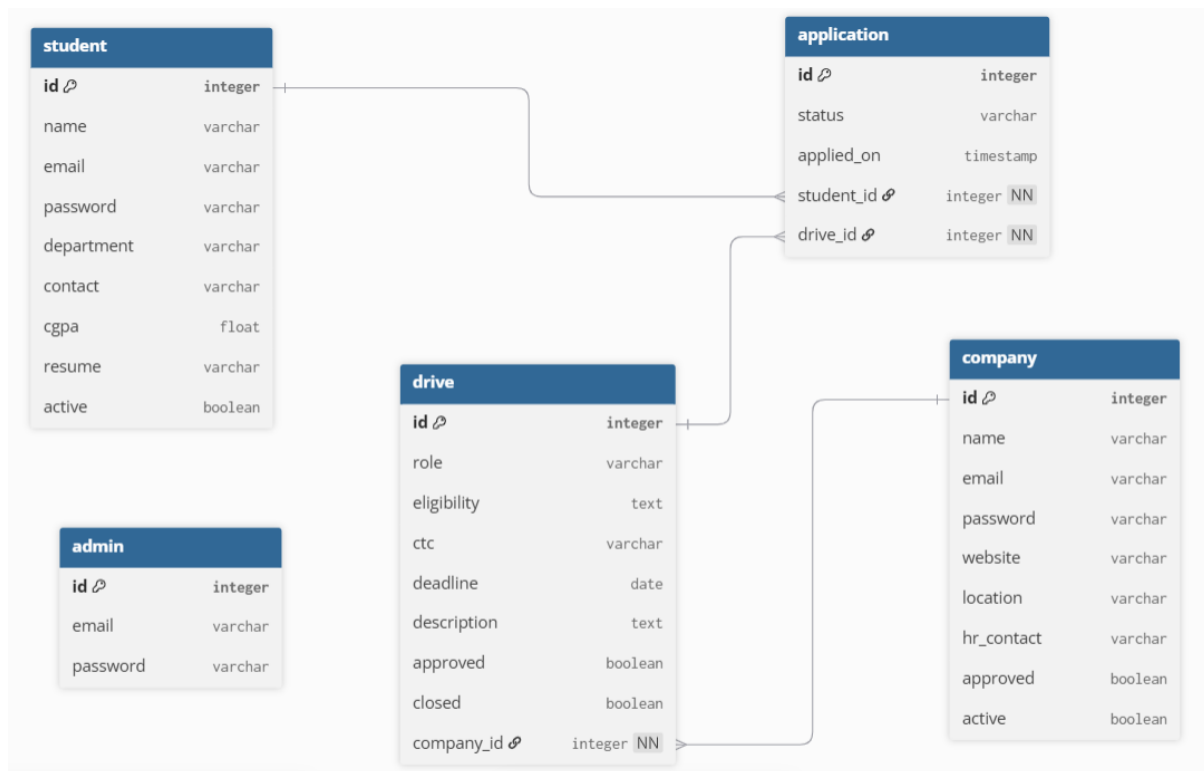
One drive can receive applications from multiple students.

4. Many-to-Many → Student ↔ Drive (via Application table)

A student can apply to many drives, and a drive can have many students.

This many-to-many relationship is implemented using the Application table as an association table.

ER DIAGRAM:



API RESOURCE ENDPOINTS:

Endpoint	Method	Description
/student/register	POST	Register a new student
/company/register	POST	Register a new company
/student/login	POST	Authenticate student
/company/login	POST	Authenticate company
/admin/login	POST	Authenticate admin
/drives	GET	Fetch all approved drives
/drives	POST	Company creates a new drive
/drives	GET	View drive details
/drives/<id>/approve	PUT	Admin approves drive
/companies/<id>/approve	PUT	Admin approves company
/applications	POST	Student applies to drive
/applications	GET	View applications
/applications/<id>/status	PUT	Update application status

ARCHITECTURE OVERVIEW:

- **app.py** – Main Flask application entry point and configuration
- **models.py** – Database models defined using Flask-SQLAlchemy
- **templates/** – Jinja2 HTML templates for UI rendering
- **static/** – CSS and Bootstrap styling files
- **database.db** – SQLite database file

IMPLEMENTED FEATURES:

- Admin, Student, and Company authentication system
- Student and Company registration
- Company drive creation and management
- Admin approval of companies and drives
- Student application submission to drives
- Application status tracking (Applied, Selected, Rejected)
- Role-based access control
- Secure password hashing
- Session management using Flask-Login
- Responsive UI built with Bootstrap

VIDEO PRESENTATION:

<https://drive.google.com/file/d/1A--NNk0FVfjCIanpnYx6rHsb04ZbY48P/view?usp=sharing>